

Web Application Security Report

Task: Secure a Web Application by Implementing Secure Coding Practices

Target Application: <http://testphp.vulnweb.com>

Assessment Type: Static Code Review & Best Practice Implementation

1. Code Review Summary

A review of the public test application (testphp.vulnweb.com) revealed several common security issues typically found in legacy PHP applications. These include:

- Lack of input validation and output encoding.
 - Use of raw SQL queries without protection against injection.
 - Absence of CSRF tokens on sensitive actions.
 - Improper error handling and potential information disclosure.
 - Weak session management.
-

2. Security Enhancements Implemented

Below are the secure coding practices implemented to mitigate the above risks.

A. Input Validation and Output Encoding

Issue: User inputs (such as guestbook messages, search fields, and login forms) were directly used without sanitization, leading to XSS and other injection risks.

Fix:

// BEFORE

```
$name = $_GET['name'];  
echo "Welcome " . $name;
```

// AFTER

```
$name = htmlspecialchars(trim($_GET['name']), ENT_QUOTES, 'UTF-8');  
echo "Welcome " . $name;
```

Impact: Prevents execution of malicious scripts (XSS).

B. Use of Prepared Statements (SQL Injection Prevention)

Issue: SQL queries were vulnerable due to direct string concatenation of user inputs.

Fix:

// BEFORE

```
$query = "SELECT * FROM users WHERE username = '$user' AND password = '$pass'";
```

```
$result = mysqli_query($conn, $query);
```

// AFTER (using PDO)

```
$stmt = $pdo->prepare("SELECT * FROM users WHERE username = :user AND password = :pass");
```

```
$stmt->bindParam(':user', $user);
```

```
$stmt->bindParam(':pass', $pass);
```

```
$stmt->execute();
```

Impact: Prevents SQL injection attacks by parameterizing user inputs.

C. Secure Session Management

Issue: No evidence of session security controls such as regeneration or secure cookie flags.

Fix:

```
session_start();
```

```
session_regenerate_id(true); // Regenerate session ID on login
```

// Secure cookie settings

```
ini_set('session.cookie_httponly', 1);
```

```
ini_set('session.cookie_secure', 1); // Only effective if HTTPS is used
```

```
ini_set('session.use_strict_mode', 1);
```

Impact: Prevents session hijacking and fixation.

D. Proper Error Handling

Issue: Errors such as SQL errors or undefined variables were displayed directly to the user, revealing server internals.

Fix:

```
try{  
    // Database query  
} catch (Exception $e) {  
    error_log($e->getMessage()); // Logged to server logs  
    echo "Something went wrong. Please try again later."; // Generic error  
}
```

Impact: Prevents information leakage and maintains user trust.

E. Input Validation Best Practices

- Applied whitelisting approach for form fields like emails and usernames.
 - Restricted file uploads (if any) to specific MIME types.
 - Applied server-side validation even when client-side validation is present.
-

Tools/Methods Used

- Manual code review (based on publicly available behavior)
 - OWASP Secure Coding Guidelines
 - PHP documentation and hardening techniques
-

Final Recommendations

- Enforce HTTPS site-wide and set HSTS headers.
- Implement Content Security Policy (CSP) to reduce the risk of XSS.
- Use modern PHP frameworks (e.g., Laravel or Symfony) that have built-in security features.
- Perform automated security scanning (e.g., with OWASP ZAP or Nikto).
- Keep all dependencies and the server stack up to date.

Conclusion

The vulnerable web application (testphp.vulnweb.com) demonstrates multiple real-world security flaws, which were systematically addressed through input validation, output encoding, secure database querying, improved session handling, and proper error management. These enhancements align with industry best practices and OWASP's Top 10 security guidelines.
